⚠️ Note: This Setup Is Not Applicable for Jekyll-Based Projects

> **The instructions below for using a Python server will not work for this project anymore because it now uses Jekyll and front matter (YAML metadata).**
>
> **To set up and run the project correctly, please refer to the `jekyll-localhost-setup.md` or `jekyll-localhost-setup.pdf` files for complete installation and local server instructions using Jekyll.**

Understanding the Need for a Python Server and Troubleshooting Header.html Linking Issues

When working with web development, it is common to test files locally in a browser. However, there are limitations when loading files, especially when using JavaScript to link HTML components like a `header.html` into the main webpage. This document explores why running a Python server resolves these issues and how to set it up.

Why Use a Python Server?

1. **CORS Restrictions:** Modern browsers, like Chrome and Edge, impose security restrictions called Cross-Origin Resource Sharing (CORS). When files are loaded locally (e.g., from `file://`), the browser blocks certain operations, such as AJAX requests or file imports, for security reasons.
2. **Dynamic Content Loading:** Using JavaScript to dynamically load parts of a webpage (e.g., loading `header.html` into the main page) requires a server to properly handle these requests. Without a server, the browser cannot interpret or serve these files correctly.
3. **Standard Practice:** Running a server, even a lightweight one like Python's built-in server, simulates a real web hosting environment, ensuring the website behaves as expected.

Steps to Set Up a Python Server

1. **Navigate to the Project Directory:** Open a terminal or command prompt and navigate to the folder containing `header.html` and other project files. Use the `cd` command to switch directories:

   ```
   cd path/to/project
   ```

2. **Start the Python Server:** For Python 3, use the following command:

```
python -m http.server 8000
```

This starts a lightweight HTTP server on port 8000. The project can then be accessed by navigating to `http://localhost:8000` in a browser.

3. **Verify the Setup:** Open a browser and go to `http://localhost:8000`. The website should load, with the `header.html` included dynamically.

How to Dynamically Link `header.html`

Here is an example of how to include `header.html` using JavaScript:

`index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Dynamic Header Example</title>
  </head>
  <body>
    <div id="header"></div>
    <script src="script.js"></script>
  </body>
</html>
```

`header.html`

```
<header>
  <h1>Welcome to the Website</h1>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">About</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </nav>
</header>
```

`script.js`

```
document.addEventListener("DOMContentLoaded", function () {
  fetch("header.html")
```

```
    .then((response) => {
      if (!response.ok) {
        throw new Error("Failed to load header.html");
      }
      return response.text();
    })
    .then((data) => {
      document.getElementById("header").innerHTML = data;
    })
    .catch((error) => {
      console.error("Error loading header:", error);
    });
});
```

Why the Header Only Works on a Python Server

When the `index.html` file is opened directly in the browser (via `file://`), the browser treats file requests as security-sensitive operations. Fetching `header.html` using JavaScript is blocked in this case.

By running a Python server, we:

- Serve the files over `http://`, making the browser treat requests as coming from a trusted source.
- Ensure that the `fetch` API can access and load `header.html` without errors.

Troubleshooting

1. **Ensure Correct Paths:** Verify that `header.html` and `script.js` are in the same directory as `index.html`, or adjust the file paths accordingly.
2. **Check Browser Console:** Open the developer tools (F12) in the browser to check for errors. If `header.html` fails to load, the console will provide details.
3. **Port Conflict:** If port 8000 is in use, try a different port by running:

   ```
   python -m http.server 8080
   ```

   Then access `http://localhost:8080`.

By using a Python server, the development environment is aligned closer to a production setup, ensuring dynamic functionality works smoothly and securely.